



UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

forward together
sonke siya phambili
saam vorentoe

DESIGN (E) 314
TECHNICAL REPORT

Embedded Driver Assistance System (E-DAS)

Author:
Lena Matota

Student Number:
24989142

April 6, 2026

Plagiaatverklaring / Plagiarism Declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektuele eiendom van ander persone asof dit jou eie werk is.

Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.

2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is.

I agree that plagiarism is a punishable offence because it constitutes theft.

3. Ek verstaan ook dat direkte vertalings plagiaat is.

I also understand that direct translations are plagiarism.

4. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelike aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is.

Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.

5. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie.

I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.

L Matota.

Handtekening / Signature

24989142

Studentenommer / Student number

LN Matota

Voorletters en van / Initials and surname

06/04/2026

Datum / Date

Contents

1	Task 1: Hardware design and implementation	3
1.1	LEDs (Debug)	3
1.2	Temperature Sensor	3
1.3	Tactile Push-Buttons	4
2	Software design and implementation	5
2.1	Button bounce handling	5
2.2	Temperature Sensor Sampling	6
3	Measurements and Results	7
3.1	LED Functionality	7
3.2	Temperature Sensor Measurement and Calibration	7
3.3	Button and Code Debouncing	8
4	End-of-Life Procedures and Environmental Impact	10

List of Figures

1	LED (Debug) Circuit	3
2	LMT01 two-pin architecture	3
3	LMT01 Circuit Implementation	4
4	Active-High Configuration for Tactile Push-Button	4
5	Button Bounce Handling Flowchart	5
6	Temperature Sensor Sampling and Measurement Flowchart	6
7	Current Verification of the LEDs via the Oscilloscope	7
8	LED Alarm Procedure Signal	7
9	Temperature Sensor Pulse Reading	8
10	Stat Command Exhibiting Temperature Alarm Condition	8
11	Button Press without Debounce Implementation	9
12	Button Press with Debounce Implementation	9

List of Abbreviations

E-DAS embedded driver assistance system

GPIO general pin input/output

ISR interrupt service routine

LEDs light-emitting diodes

PCB print circuit boards

WEEE waste electrical and electronic equipment

1 Task 1: Hardware design and implementation

1.1 LEDs (Debug)

The light-emitting diodes (LEDs) used in the system are 3.0 mm round type lamps. They are connected to a 3.3 V supplied from the STM32F411RE Nucleo board [1]. The datasheet for the LEDs specify a typical forward voltage of 2 V and a recommended operating current of 20 mA [2].

To design for safety, a current-limiting resistor is implemented to drop the excess voltage from the supply (Nucleo board) and limits the current to a safe operating level. The aforementioned parameters are used to calculate the value of the resistor, as seen in Equation 3 below.

$$R = \frac{V_{supply} - V_f}{I_f} \quad (1)$$

$$R = \frac{3.3 - 2}{0.02} \quad (2)$$

$$R = 65 \Omega \quad (3)$$

The closest approximate resistance value was 100 Ω . The circuit was implemented with a resistor of that value as shown in Figure 1 below.

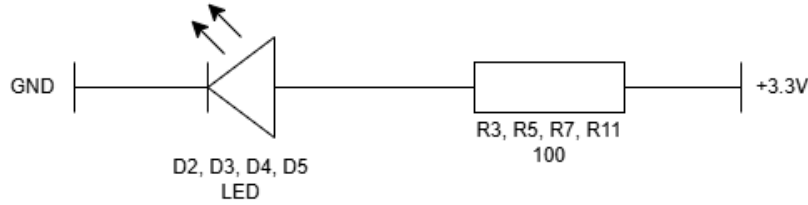


Figure 1: LED (Debug) Circuit

1.2 Temperature Sensor

The temperature sensor used for the system is an LMT01 temperature sensor. The sensor has two-pin architecture as shown in Figure 2 below.



Figure 2: LMT01 two-pin architecture

The VP receives power from the system for the temperature sensor to operate [3]. The VN pin transmits temperature data as a stream of current pulses to the microcontroller via a general pin input/output (GPIO) [3]. A bias resistor is connected to the VN pin for the purposes of [3]:

- Converting the current pulses into voltage pulses
- Creating readable logic levels for the microcontroller to process

The temperature sensor has an output current that varies from $34\mu\text{A} - 125\mu\text{A}$ [3]. The microcontroller datasheet specifies the following VIL and VIH levels, assuming $V_{DD} = 3.3\text{V}$ (Equation 4 and Equation 5) [1]:

$$V_{IL} = 0.3V_{DD} = 0.99\text{V} \quad (4)$$

$$V_{IH} = 0.7V_{DD} = 2.31\text{V} \quad (5)$$

Using this information, a range of resistor values was calculated to determine the bias in Equation 6 and Equation 7 [1]:

$$R_H = \frac{0.99}{34 \times 10^{-6}} = 29.12\text{ k}\Omega \quad (6)$$

$$R_L = \frac{2.31}{125 \times 10^{-6}} = 18.48\text{ k}\Omega \quad (7)$$

A resistor value of $27\text{ k}\Omega$ was implemented as shown in Figure 3 below. It must also be noted that there were issues with the 3.3V powering the temperature sensor. This was because the minimum voltage difference of 2V across the VP and VN pin was not achievable with the 3.3V . As a result, the 5V was used and the temperature sensor was operational.

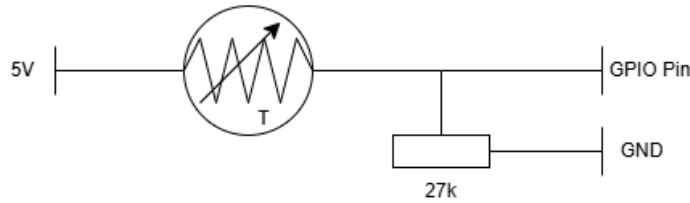


Figure 3: LMT01 Circuit Implementation

1.3 Tactile Push-Buttons

The tactile push-buttons were required to be built in an active-high configuration as shown in Figure 4 below.



Figure 4: Active-High Configuration for Tactile Push-Button

The maximum rating conditions for the buttons is given as 12V and 50mA [4]. This meant that the button could be connected powered by the 3.3V power supply and a GPIO pin from the microcontroller could receive the button press signal directly. This means that when the switch is open, the pull-down resistor pulls the output signal, sent to the GPIO pin, low. This also means that when the switch is pushed closed, the output signal to the GPIO pin is pulled high to be registered by the microcontroller. It must be noted that due to lack of space, the pull-down resistor was not physically implemented. Instead, it was implemented in software when configuring the GPIO in with a pull-down resistor, [1].

2 Software design and implementation

2.1 Button bounce handling

As stated before in section 1.3, the tactile push-buttons are to be configured in an active-high configuration. Each button was connected to the microcontroller via a GPIO that was configured as a GPIO_Input, pulled low via an internal pull-down resistor.

The system was required to respond correctly to button presses within 200 ms, while ignoring false signals caused by mechanical contact bounce [5]. Mechanical bounce occurs because the physical contacts of a button briefly make and break connection multiple times once pressed [6]. If this issue is left unchecked, a single press can register as many rapid presses in software, which can cause faulty execution of code such as toggling an LED multiple times or triggering alarms erroneously.

The requirements for debouncing include:

- Ignore repeated interrupts from the same button that occur within a defined debounce window.
- Detect only the first valid rising edge of a button press.
- Not block execution (no HAL_Delay() calls within the interrupt).
- Support all five buttons independently.

The debouncing approach chosen is interrupt-driven timestamping [6]. It is also known as "last-tick" debouncing [6]. This method was selected over polling-based debouncing because the interrupt-driven method uses the STM32's external interrupt (EXTI) system. As shown in the flowchart in Figure 5, the interrupt fires on a rising edge, but only sets a flag if enough time has elapsed since the last accepted press. This, however, does not halt the main loop.

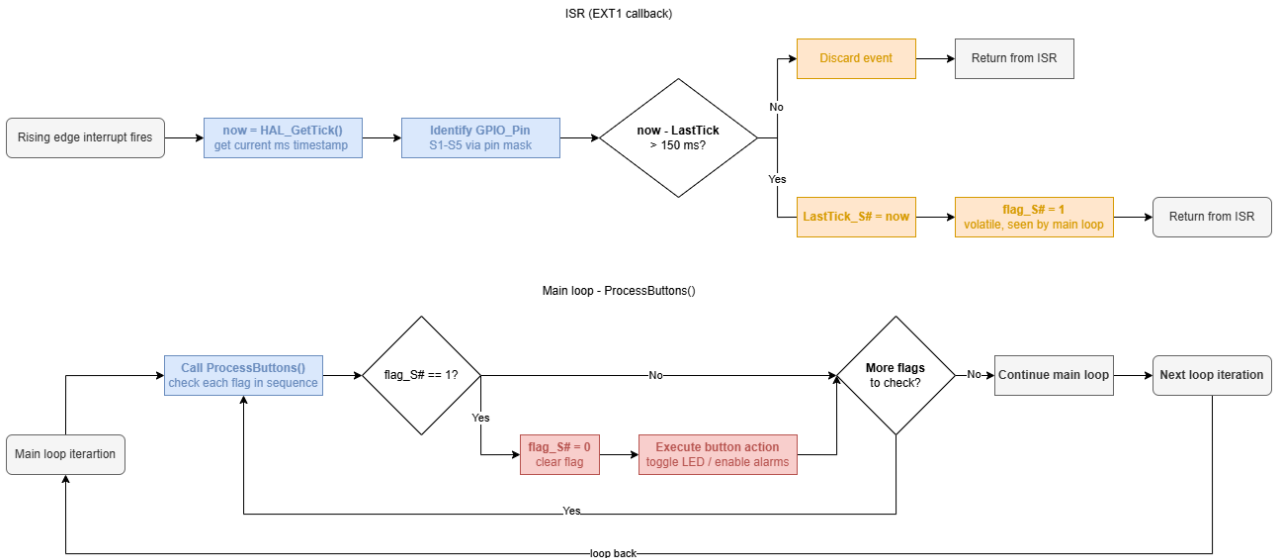


Figure 5: Button Bounce Handling Flowchart

This allows each button to maintain its own LastTick variable, so that debouncing one button does not affect others. In addition, the HAL_GetTick() function provides a 1 ms resolution timestamp via the SysTick timer. This timer is already configured by the HAL library [1] and requires no additional timer configuration.

Note that all five button GPIO pins are configured as GPIO_MODE_IT_RISING. This means that the pins were configured as interrupts that get triggered on the rising edge only. The same pins were also configured with GPIO_PULLDOWN to activate an internal pull-down resistor to ensure stable LOW state when the button is not pressed [1]. The EXTI interrupt lines are enabled via HAL_NVIC_EnableIRQ() [1].

A debounce guard time of 150 ms was chosen. This is larger than the typical mechanical bounce duration (5–20 ms), but short enough to allow rapid intentional presses to register. The debounce time rejects bounces without limiting response speed for real user presses.

In the `HAL_GPIO_EXTI_Callback()` function, when a rising-edge interrupt fires on any button pin, the current system tick (`now = HAL_GetTick()`) is compared to the last accepted tick for that button. If the time difference exceeds 200 ms, the press is accepted, which means the `LastTick` variable is updated and the corresponding flag (`flag_S1` through to `flag_S5`) is set to 1. Otherwise the interrupt is discarded.

In the main loop, `ProcessButtons()` is called in the while loop. It checks each flag, clears it, and executes the appropriate action (e.g. toggling an LED state or enabling alarm checking). This two-stage design, in which a flag is set in an interrupt service routine (ISR) and action is taken in the main loop to ensure ISRs remain short and decisive.

2.2 Temperature Sensor Sampling

The LMT01 digital temperature sensor communicates via a pulse-train protocol, in which, within a fixed timing window, it outputs a burst of pulses where the total pulse count encodes the measured temperature [3]. The conversion formula is below:

$$T(^{\circ}C) = \left\lceil \frac{\text{pulse_count}}{4096} \times 256 \right\rceil - 50 \quad (8)$$

As shown in the flowchart in Figure 6, the system must read a new temperature value at a defined interval, convert the raw pulse count to $^{\circ}C$, and flag an alarm if the temperature exceeds $30^{\circ}C$ (SS6).

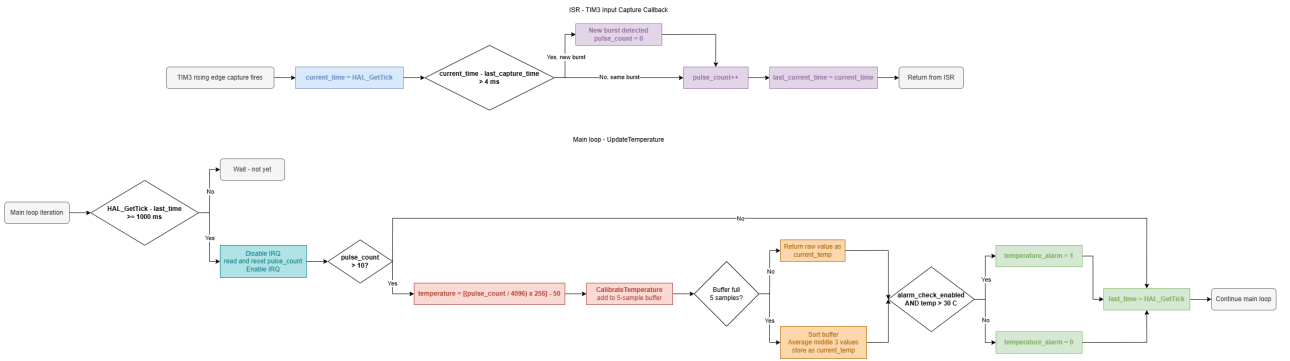


Figure 6: Temperature Sensor Sampling and Measurement Flowchart

TIM3 is configured as an Input Capture timer on Channel 2 which is to be triggered on rising edges. Each rising edge from the LMT01 data line fires the `HAL_TIM_IC_CaptureCallback()` function, where the inter-pulse gap is checked. If more than 4 ms has passed since the last captured edge, this signals the start of a new burst and the pulse counter (`pulse_count`) is reset. Otherwise, the count is incremented.

Every 1000 ms, the `UpdateTemperature()` function reads and resets `pulse_count`, applies the conversion formula, and passes the result to `CalibrateTemperature()`. This is done with the interrupts briefly disabled to prevent a race condition¹. This function implements a 5-sample median-average filter. The function stores the last five readings in a circular buffer, sorts them, and returns the mean of the middle three values. This removes occasional outliers caused by noise or missed pulses and the filtered result is stored in `current_temp`.

¹A race condition occurs when two or more parts of a program try to access and modify the same variable at the same time. The final result depends on the order in which they execute which is unpredictable. [6]

3 Measurements and Results

3.1 LED Functionality

To verify whether the LED circuits met the design requirements stipulated for the project, the measured current through each LED must be confirmed to be within the recommended operating range specified in the datasheet (20 mA). This was done by measuring the voltage difference over the LED circuit using an oscilloscope, as seen in Figure 7.

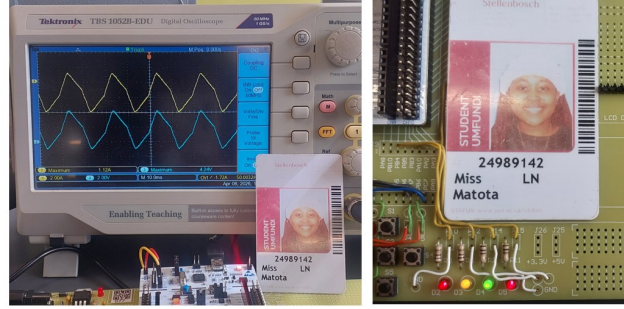


Figure 7: Current Verification of the LEDs via the Oscilloscope

This voltage difference was used to determine the current through the LED circuit as seen in Equation 9

$$I = \frac{V_{channel1} - V_{channel2}}{R} = \frac{4.24 - 1.12}{100} = 3.12 \text{ mA} \quad (9)$$

This current is within the recommended operating current of the LED as stipulated by the datasheet [2]. The flashing behaviour of the LEDs was verified by enabling each alarm condition individually by running the code and triggering each alarm respectively, and observing the corresponding LED response. An LED was connected to an oscilloscope and the signal was measured, confirming that each LED flashed at approximately 500 ms ON and 500 ms OFF as required by SS7 [5], as in Figure 8.

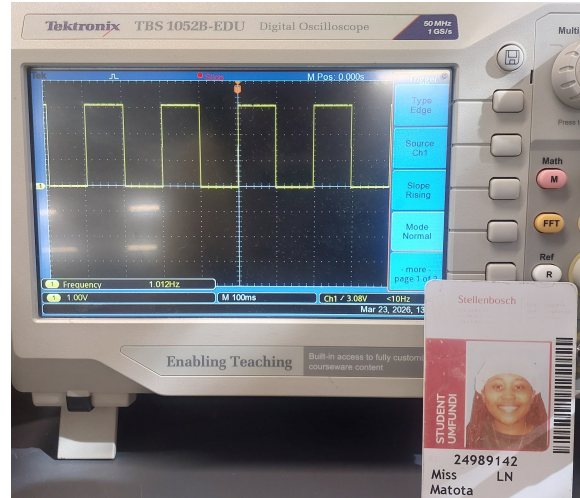


Figure 8: LED Alarm Procedure Signal

3.2 Temperature Sensor Measurement and Calibration

To verify the temperature sensor circuit, the LMT01 was connected as per the circuit shown in Figure 3, with the 27 kΩ bias resistor on the VN pin and the 5 V supply on VP. The digital pulse output on the VN pin was observed using an oscilloscope to confirm that a pulse train was being generated by the sensor, as shown in Figure 9.

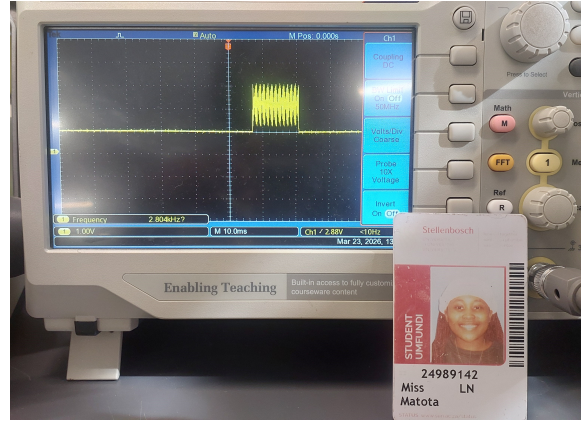


Figure 9: Temperature Sensor Pulse Reading

The pulse burst was visible as a series of regular pulses followed by a silent window period, consistent with the LMT01 datasheet specification [3]². The temperature reading reported via UART was compared against a reference thermometer placed in the same environment. At room temperature (approximately 24 °C), the system reported a temperature value within 1 °C of the reference. This confirmed that the conversion formula and the 5-sample median-average filter were operating correctly. To test the alarm threshold, the sensor was gently warmed by holding it between fingers until the reported temperature exceeded 30 °C, at which point the D5 LED began flashing and the UART @Stat response reflected High Temperature: 1, confirming correct alarm detection as required by SS6, as seen in Figure 10.

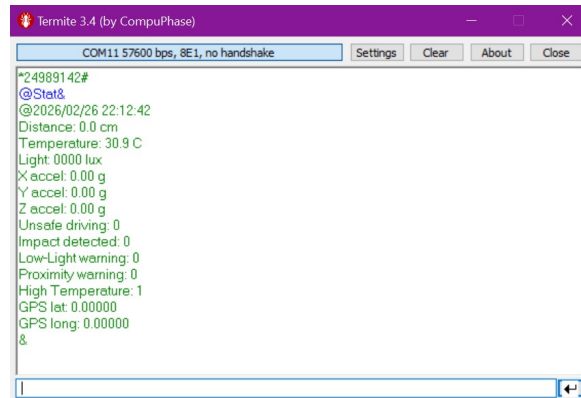


Figure 10: Stat Command Exhibiting Temperature Alarm Condition

3.3 Button and Code Debouncing

To verify button debouncing, each of the five tactile push-buttons (S1–S5) was pressed repeatedly in quick succession while the corresponding LED state was observed. Without debouncing, rapid mechanical bounce would cause the LED to toggle multiple times per press. With the 200 ms debounce guard implemented in HAL_GPIO_EXTI_Callback(), each button press produced exactly one LED toggle. This confirmed that the debouncing logic was rejected false bounce signals correctly. To further verify the debounce timing, an oscilloscope was connected to the GPIO input pin of button S1 (PA0) while the button was pressed. The raw signal showed multiple transitions within approximately 5–15 ms of the initial press, consistent with typical mechanical bounce behaviour, as seen in Figure 11.

²The oscilloscope confirmed a pulse burst was present on the VN pin, consistent with the LMT01 pulse-train protocol. The frequency measurement is displayed with a question mark, which represents an uncertainty indicator, due to the non-continuous nature of the burst signal.

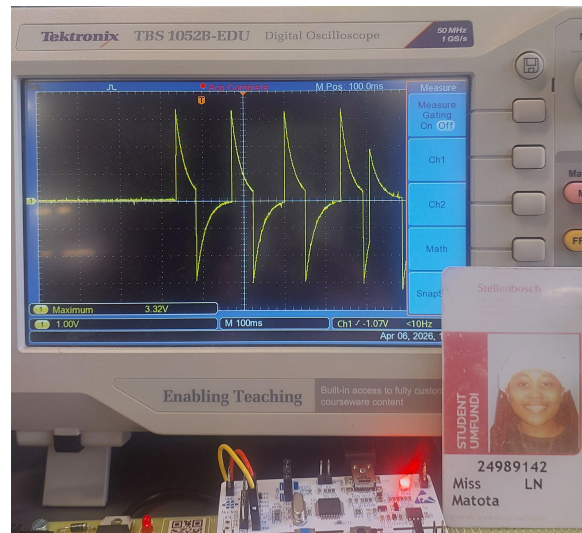


Figure 11: Button Press without Debounce Implementation

The software correctly ignored all transitions after the first within the 200 ms window, as confirmed by the LED toggling only once per deliberate press, as seen in Figure 12.

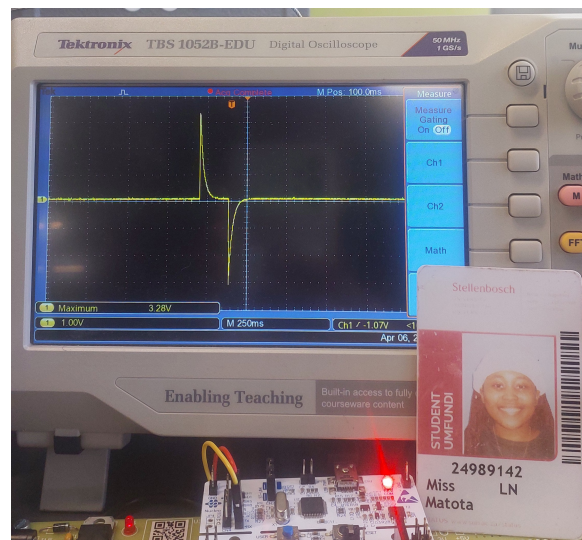


Figure 12: Button Press with Debounce Implementation

4 End-of-Life Procedures and Environmental Impact

The electronic components used in the embedded driver assistance system (E-DAS) project all fall under the category of waste electrical and electronic equipment (WEEE). Improper disposal of these components can pose a great environmental risk that must be considered as part of responsible engineering practice.

The primary environmental concern with electronic components is the presence of hazardous materials. print circuit boards (PCB) and integrated circuits can contain lead (found in solder), cadmium, mercury, brominated flame retardants, and hexavalent chromium [7]. When these materials are discarded of in general landfill, they can leach into soil and groundwater over time. This can cause contamination that is harmful to ecosystems and human health. LEDs contain semiconductor materials such as gallium and arsenic compounds, which are toxic if released into the environment [8]. These restrictions on hazardous substances in electrical and electronic equipment are formally recognised under the Restriction of Hazardous Substances (RoHS) Directive, which informed similar legislation globally [8].

In South Africa, the electronic waste management is governed by the National Environmental Management: Waste Act (Act 59 of 2008) and its associated Extended Producer Responsibility (EPR) regulations, which came into effect in November 2021 [9]. Under these regulations, producers, importers, and retailers of electrical and electronic equipment are legally obligated to register with a Producer Responsibility Organisation (PRO) and ensure that end-of-life products are collected and recycled through approved channels [9]. Students and institutions discarding electronic components are encouraged to make use of certified e-waste collection points instead of general waste facilities [9]. Organisations such as e-Waste Africa, Rapid Phase E-Waste Recycling, and the ewasa) operate certified collection and recycling facilities, [10, 11]. Components should be delivered to these facilities rather than placed in general waste bins. Some municipalities also operate scheduled e-waste drop-off days. Stellenbosch University’s own facilities management also provide guidance on the correct disposal of laboratory electronic waste.

Methods of disposal that are explicitly not permitted include placing electronic components in general household or laboratory waste bins, burning components (which releases toxic fumes including dioxins), disassembling components without protective equipment, and discarding components in water drains or open ground [9]. Exportation of e-waste to developing countries under the guise of "second-hand goods" is also prohibited under the Basel Convention, to which South Africa is a party to the agreement [12].

To maximise sustainable engineering practice, components that are still functional should be returned to the departmental stock for reuse in future projects [9]. Where components are damaged, they should be separated by material type where possible such as metal, plastic, PCB, to facilitate more efficient recycling [9]. Overall, the preferred order of end-of-life actions is: reuse, then certified recycling, with landfill being an absolute last resort that should be avoided entirely for electronic components.

References

- [1] STMicroelectronics, “Stm32f411xc stm32f411xe,” 2024, accessed: 2026-02-18.
- [2] B. E. Limited, “Approval sheet,” accessed: 2026-02-18.
- [3] T. Instruments, “Lmt01 0.5°c accurate 2-pin digital output temperature sensor with pulse count interface,” <https://stemlearn.sun.ac.za/mod/folder/view.php?id=170446>, accessed: 2026-02-20.
- [4] “Tact switch specification,” <https://stemlearn.sun.ac.za/mod/folder/view.php?id=170446>, accessed: 2026-02-20.
- [5] *Design (E) 314 Project Definition*, <https://stemlearn.sun.ac.za/course/section.php?id=45314>, 2026.
- [6] G. Wright, “Definition: Debouncing,” *TechTarget*, 2018.
- [7] U. N. E. Programme, “Solving the e-waste problem: A synthesis of possible measures,” UNEP, Tech. Rep., 2009, accessed: 2026-04-01. [Online]. Available: <https://www.unep.org/resources/report/solving-e-waste-problem-synthesis-possible-measures>
- [8] E. Parliament, “Restriction of hazardous substances in electrical and electronic equipment (rohs),” *Official Journal of the European Union*, 2011, accessed: 2026-04-01. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32011L0065>
- [9] Republic of South Africa, “National environmental management: Waste act,” Pretoria, 2008.
- [10] e-Waste Association of South Africa, “e-waste association of south africa (ewasa),” 2024, accessed: 2026-04-01. [Online]. Available: <https://ewasa.org>
- [11] e Waste Africa, “e-waste africa recycling,” 2024, accessed: 2026-04-01. [Online]. Available: <https://ewasteafrica.co.za>
- [12] B. C. Secretariat, “Basel convention on the control of transboundary movements of hazardous wastes and their disposal,” 1989, accessed: 2026-04-01. [Online]. Available: <https://www.basel.int/TheConvention/Overview/tabid/1271/Default.aspx>